

Docker

Docker

- Les clusters
 - Les orchestrateurs
 - Installer Docker machine
 - Mise en place d'un cluster Swarm
 - Déploiement de services sur le cluster
 - Le registre privé
 - Les interfaces graphiques
 - Portainer.io
 - Swarmpit
 - Les contraintes de placement
- Traefik

Les clusters

Les orchestrateurs

Permet de :

- provisionner et placer les conteneurs
- monitorer
- gestion du failover
- dimensionnement
- gestion des mises à jour et rollbacks des conteneurs
- découverte de service et gestion du réseau

Kubernetes

Développé par Google

Adopter aux déploiements d'envergure

Fonctionnalités très avancées (autoscaling, load balancing..)

Prise en main difficile

Swarm

Natif / facile à mettre en oeuvre

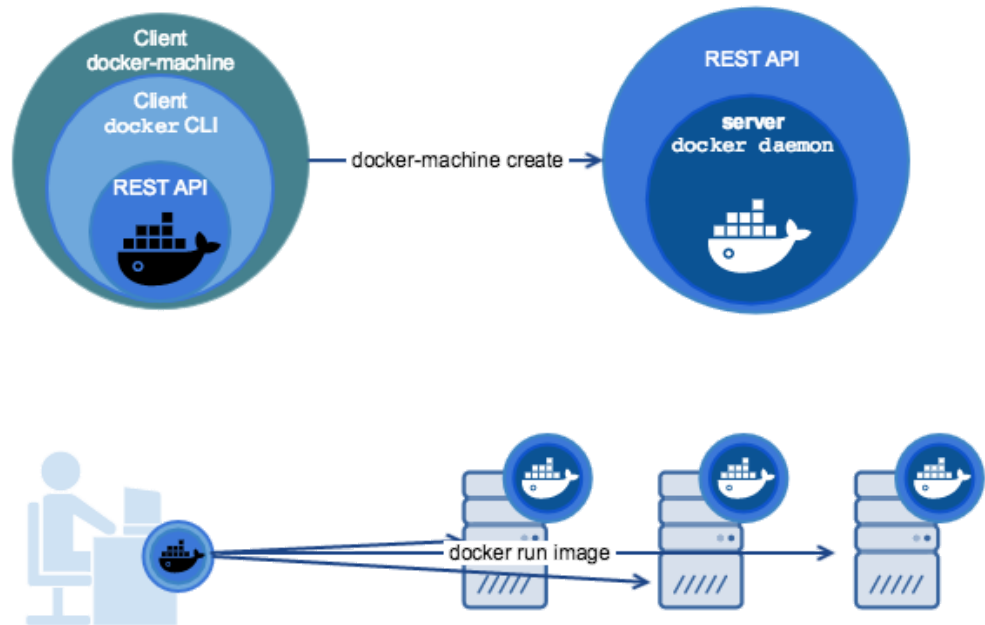
Fonctionnalités limités mais souvent suffisante

Orchestrateurs packagées

Les cloud providers proposent désormais des orchestrateurs "As A Service". (Scaleway, ECS, AKS, GCE)

Installer Docker machine

<https://docs.docker.com/machine/>



Installation par binaire : <https://docs.docker.com/machine/install-machine/>

Sur Linux :

```
curl -L https://github.com/docker/machine/releases/download/v0.16.0/docker-machine-$(uname -s)-$(uname -m) >/tmp/docker-machine
sudo mv /tmp/docker-machine /usr/local/bin/docker-machine
chmod +x /usr/local/bin/docker-machine
```

Tester l'installation :

```
docker-machine version
```

Listez les machines

```
docker-machine ls
```

Créer une machine avec Virtualbox

```
docker-machine create --driver virtualbox mondocker1
```

Affichage des variables d'environnement de connexion à la machine :

```
docker-machine env mondocker1
```

Basculer sur la machine :

```
eval $(docker-machine env mondocker1)
docker ps
```

Machine actuellement manipulé

```
docker-machine active
```

IP de la machine

```
docker-machine ip mondocke1
```

Prendre la main en ssh

```
docker-machine ssh mondocke1
```

Autres commandes :

```
docker-machine start mondocke1
docker-machine stop mondocke1
docker-machine status mondocke1
```

Mise en place d'un cluster Swarm

<https://docs.docker.com/engine/swarm/>

```
# initialisation du cluster
docker swarm init --listen-addr [ip leader1] --advertise-addr [ip leader1]

# rejoindre le cluster
docker swarm join-token -q worker
# sur le noeud à lier
docker swarm join --token $token [ip leader1]:2377

# affiche les infos de notre docker engine dont les infos du cluster swarm
docker info

# sur le leader, affiche la liste des noeuds
docker node ls

# affiche la liste des réseaux
docker network ls

# Modifier la disponibilité d'un noeud
docker node update --availability drain leader1

# forcer la redistribution
docker service update --force [service name]

# Aide sur la commande de gestion des services
docker service help

# dimensionner manuellement un service
docker service scale [service name]=3
```

Déploiement de services sur le cluster

```
# déploiement d'une stack
docker stack deploy mastack --compose-file docker-compose.yml

# affichage des stacks déployées
docker stack ls

# affichage des services d'une stack
docker stack services mastack

# affichage des tâches pour une stack
docker stack ps mastack
```

Nouveautés pour le `docker-compose.yml`, à mettre pour chaque service :

```
deploy:
  replicas: 5
  restart_policy:
    condition: on-failure
  resources:
    limits:
      cpus: "0.1"
      memory: 50M
```

<https://docs.docker.com/compose/compose-file/compose-file-v3/#deploy>

Pour déployer une stack avec des services dont les images sont à builder, il faut que celle-ci soit disponible depuis tous les noeuds

Exercice

Déployer la stack avec Adminer et mariadb sur le cluster.

Vérifier que les conteneurs communiquent bien, même si ils ne sont pas sur le même noeud.

Le registre privé

Lancer le service

```
version: '3.4'

services:
  apitest:
    image: registry:2
    ports:
      - 5000:5000
```

Autoriser les "insecure" registries dans `/etc/docker/daemon.json` et redémarrer les démons Docker:

```
{  
  "insecure-registries": ["192.168.56.121:5000"]  
}
```

Build and push :

```
docker build -t api_test .  
docker tag api_test 192.168.56.121:5000/api_test  
docker push 192.168.56.121:5000/api_test
```

Listez les images disponibles :

```
curl http://192.168.56.121:5000/v2/_catalog
```

Exercice

Créer un registry privé et pousser l'image avec l'API Python dessus, puis déployer la stack avec cette image et le redis.

Les interfaces graphiques

Portainer.io

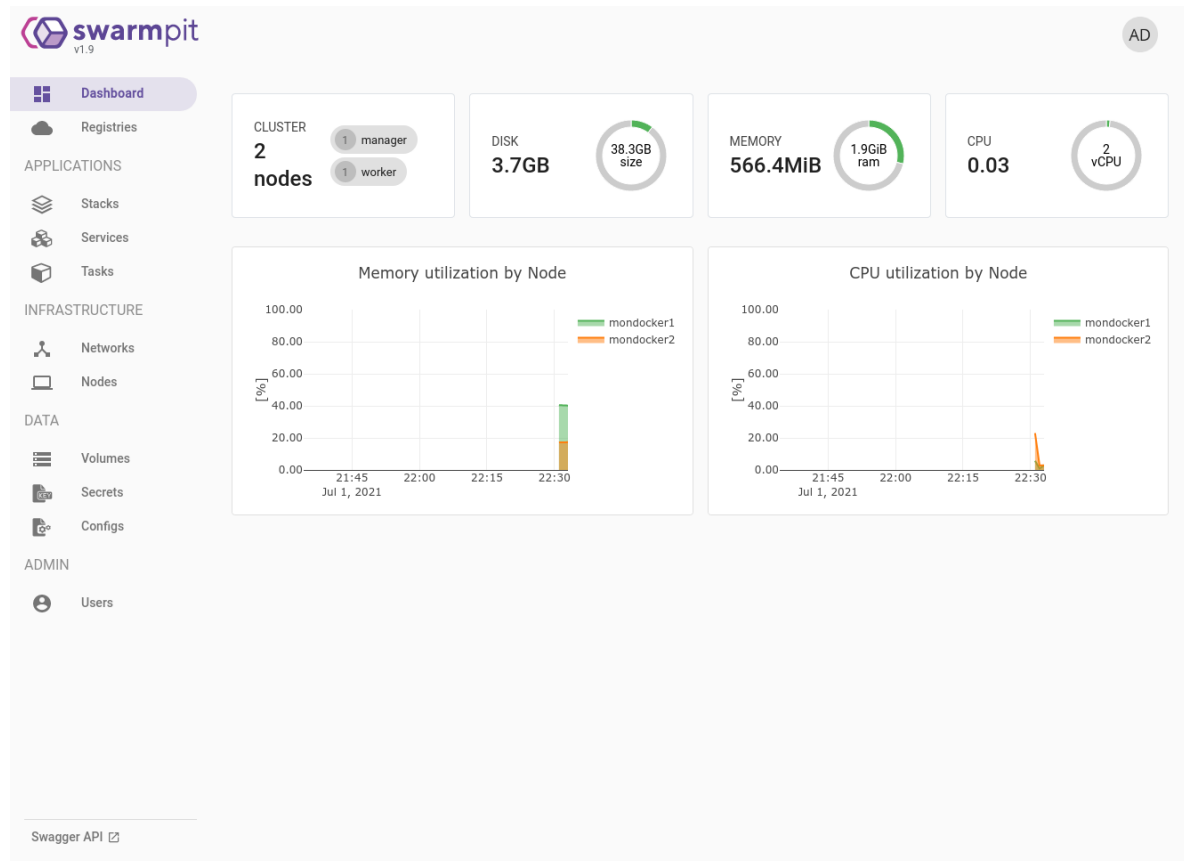
The screenshot displays the Portainer.io Swarm visualizer interface. On the left is a dark sidebar with navigation links: Home, PRIMARY, Dashboard, App Templates, Stacks, Services, Containers, Images, Networks, Volumes, Configs, Secrets, Swarm, SETTINGS, Users, Endpoints, Registries, and Settings. The main area is titled 'Swarm visualizer' and shows 'Cluster information' and 'Cluster visualizer'. The cluster consists of two nodes: 'mondocker1' (manager) and 'mondocker2' (worker). The manager node has a 'ready' status and is running several services: 'pmawithdb_mariadb' (Image: mariadb:latest, Status: running, Update: 2021-07-02 00:09:59), 'pmawithdb_mariadb' (Image: mariadb:latest, Status: rejected, Update: 2021-07-02 00:06:43), 'pmawithdb_phpmyadmin' (Image: phpmyadmin/phpmyadmin:latest, Status: running, Update: 2021-07-02 00:06:53), 'pmawithdb_phpmyadmin' (Image: phpmyadmin/phpmyadmin:latest, Status: rejected, Update: 2021-07-02 00:06:48), 'pmawithdb_phpmyadmin' (Image: phpmyadmin/phpmyadmin:latest, Status: failed, Update: 2021-07-02 00:06:43), and 'portainer_agent' (Image: portainer/agent:latest). The worker node also has a 'ready' status and is running 'pmawithdb_mariadb' (Image: mariadb:latest, Status: shutdown, Update: 2021-07-02 00:07:32) and 'portainer_agent' (Image: portainer/agent:latest, Status: running, Update: 2021-07-02 00:43:41). The top right corner shows the user 'admin' with a 'my account' link and a 'log out' button.

<https://documentation.portainer.io/>

Installation sur Docker Swarm

```
curl -L https://downloads.portainer.io/portainer-agent-stack.yml -o portainer-agent-stack.yml  
  
docker stack deploy -c portainer-agent-stack.yml portainer
```

Swarmpit



<https://swarmpit.io/>

```
docker run -it --rm \
  --name swarmpit-installer \
  --volume /var/run/docker.sock:/var/run/docker.sock \
  swarmpit/install:1.9
```

Exercice

Déployer une interface graphique sur son cluster swarm.

Visualiser le positionnement des conteneurs et leurs usages des ressources.

Les contraintes de placement

Ajouter des étiquettes aux noeuds :

```
docker node update --label-add job=worker docker2
```

Ajouter les contraintes dans le `docker-compose` :

```

version: '3.4'

services:
  apitest:
    image: 192.168.56.121:5000/api_test
    environment:
      HOST: "{{ .Service.Name }} : {{ .Node.Hostname }} -> {{ .Task.Name }}"
    ports:
      - 5000:5000
    deploy:
      replicas: 3
      placement:
        constraints: [node.labels.job == worker]

```

Exercice

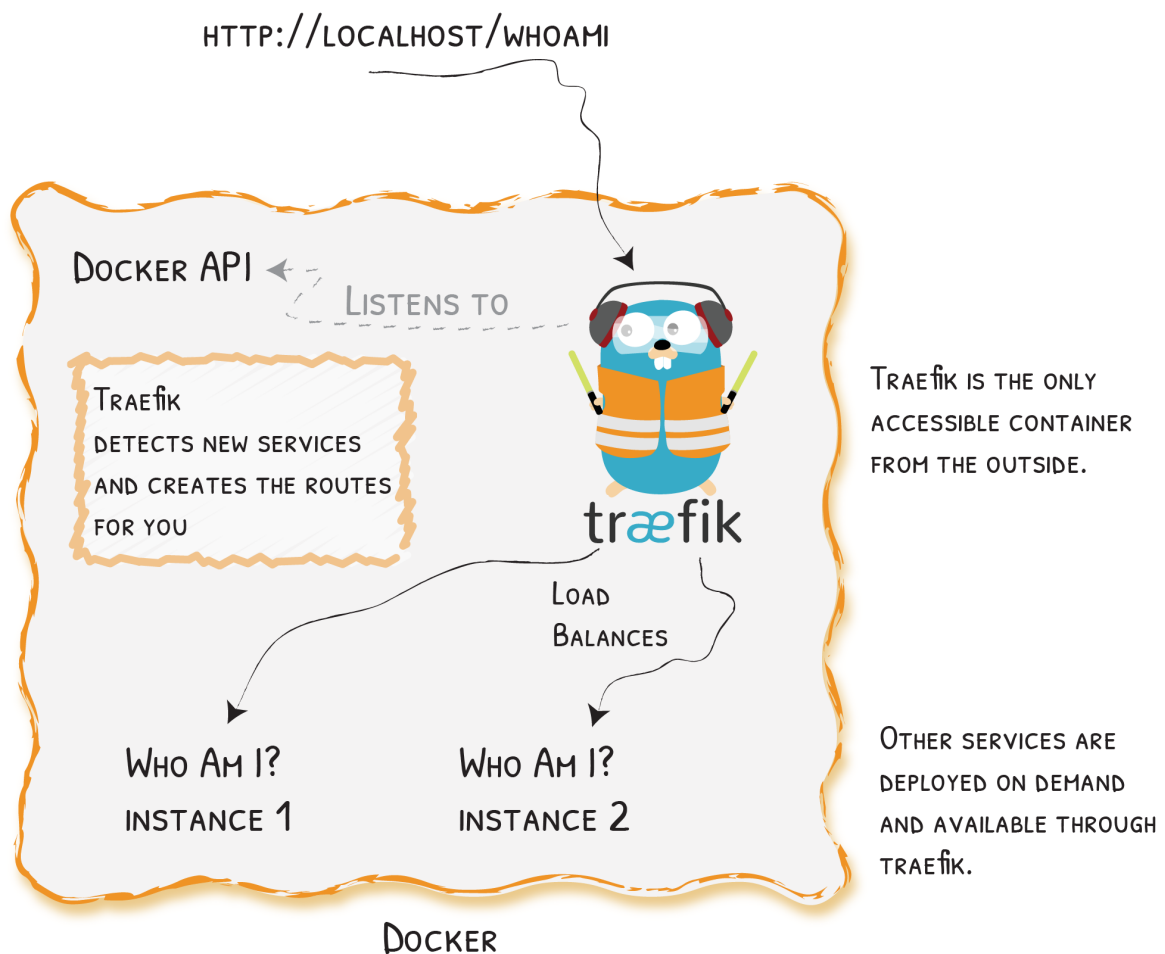
Ajouter un label de votre choix à un seul noeud et forcer un service à se déployer dessus.

En mettre un deuxième avec ce même label et arrêter l'autre noeud.

Redémarrer ensuite le noeud et forcer la redistribution.

Traefik

<https://doc.traefik.io/traefik/getting-started/quick-start/>



```

version: '3'

```

```
services:
  reverse-proxy:
    # The official v2 Traefik docker image
    image: traefik:v2.4
    # Enables the web UI and tells Traefik to listen to docker
    command: --api.insecure=true --providers.docker
    ports:
      # The HTTP port
      - "80:80"
      # The Web UI (enabled by --api.insecure=true)
      - "8080:8080"
    volumes:
      # So that Traefik can listen to the Docker events
      - /var/run/docker.sock:/var/run/docker.sock
  whoami:
    # A container that exposes an API to show its IP address
    image: traefik/whoami
    labels:
      - "traefik.http.routers.whoami.rule=Host(`whoami.docker.localhost`)"
```